

# PYTHON & SOFTWARE ENGINEERING

A Concept Reference Report

---

Prepared by

**Pranay Hajare**

September 2026

## Table of Contents

1. Indentation and Comments
2. Functions
3. Object-Oriented Programming Foundations
4. File Handling
5. Exception Handling
6. NumPy
7. Pandas
8. Selenium Automation
9. Regular Expressions
10. Multithreading, Multiprocessing, and Concurrency vs Parallelism
11. Software Development Life Cycle (SDLC)
12. Agile Philosophy
13. Scrum Framework
14. Git Version Control
15. Documentation Standards
16. Risk Management
17. Coding Standards
18. PEP 8
19. Docstrings vs Comments
20. Logging and Auditing
21. Writing Efficient Code
22. Architecture Principles — DRY, KISS, YAGNI, SOLID
23. Unit Testing
24. Ruff Linter
25. Black Formatter

*This report consolidates core working notes across Python fundamentals, key libraries, automation, and software engineering practice — covering language mechanics, data tooling, concurrency, the development lifecycle, and the standards and tools that keep a codebase maintainable.*

## 1. Indentation and Comments

---

Python is one of the few widely used languages that treats indentation as syntax rather than as a cosmetic choice. Where languages like C or Java use curly braces to mark the start and end of a block, Python relies on consistent whitespace — four spaces per level is the accepted convention. This forces every codebase to share the same visual structure no matter who wrote it.

Mixing tabs and spaces inside the same block is not tolerated. Rather than guessing which one the programmer meant, the interpreter raises an `IndentationError` immediately, trading a moment of frustration for the certainty that ambiguous whitespace never silently changes program behaviour.

Comments are most useful when they explain something the code cannot say on its own — the reasoning behind a threshold, a deliberately ignored edge case, or a workaround for a bug in a third-party library. A comment that just restates the line above it (`counter += 1 # add one`) adds clutter rather than clarity. Python has no dedicated multi-line comment syntax; a floating triple-quoted string is commonly used instead, since the interpreter evaluates it and simply discards the value.

```
if score >= 40:    # business rule: passing threshold
    status = "Pass"
else:
    status = "Fail"
```

## 2. Functions

---

A function's main job is to stop logic from being duplicated. As soon as the same few lines appear in two places, that block is a strong candidate to be extracted into a function with a name describing what it does, not the mechanics of how it does it.

Parameters are the variables a function agrees to accept; arguments are the actual values supplied when it is called. Python supports positional arguments (order matters), default arguments (used when nothing is supplied), and keyword arguments (named explicitly, so order no longer matters). A function that returns a value is generally easier to test and reuse than one that only prints, because `return` hands data back to the caller while `print()` just writes to the terminal.

`*args` and `**kwargs` handle situations where the number of incoming arguments is not known in advance. `*args` gathers extra positional values into a tuple, and `**kwargs` gathers extra named values into a dictionary. Both show up constantly in decorators, wrapper functions, and flexible APIs.

```
def calc_net(gross, tax=0.2):
    return gross * (1 - tax)
```

```
def describe(*args, **kwargs):
    print(args)      # tuple of positional extras
    print(kwargs)    # dict of keyword extras
```

### 3. Object-Oriented Programming Foundations

---

A class is a blueprint rather than something usable on its own — much like a house blueprint describes where the rooms go without being a house anyone can live in. An object is the actual thing built from that blueprint, holding real values in memory. Every object created from the same class shares the same structure but can carry entirely different data.

A class contains attributes (the data it carries) and methods (the functions that act on that data). The constructor, `__init__`, runs automatically whenever a new object is created and is typically where starting attributes are set using `self`, which refers to the specific object being built rather than any other instance of the same class.

Encapsulation, inheritance, and polymorphism are what make object-oriented design genuinely useful. Encapsulation keeps data and the logic that manipulates it together, so state doesn't leak unpredictably across a codebase. Inheritance lets a child class reuse a parent's attributes and methods instead of rewriting them. Polymorphism allows the same method name to behave differently depending on the object it is called on — a `speak()` method might mean something different for a `Dog` than for a `Human`, without the calling code needing to know which one it holds.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def give_raise(self, pct):
        self.salary *= (1 + pct)

e = Employee("Riya", 50000)
e.give_raise(0.1)
```

### 4. File Handling

---

Opening a file gives a program a live connection to something on disk, and that connection needs to be closed when the work is done — otherwise the file can stay locked, buffered writes can be lost, or handles can leak in a long-running program. The `with` statement (a context manager) guarantees that closing happens automatically, even if an exception interrupts the read or write.

The mode a file is opened in changes what is allowed: `'r'` for reading (the file must already exist), `'w'` for writing (creates a new file or silently overwrites an existing one), `'a'` for appending without disturbing existing content, and `'x'` for exclusive creation, which fails loudly instead of overwriting anything that already exists.

```
with open("data.csv", "r") as f:
    records = f.read() # file is guaranteed closed here, even on error
```

## 5. Exception Handling

---

Not every error can be handled. A `SyntaxError` or an `IndentationError` means the code cannot even be parsed, so there is nothing to catch. Exceptions are different — problems that arise while otherwise valid code is running, such as dividing by zero, a missing file, or converting "abc" to an integer. Because the code itself is valid, Python provides a way to anticipate and recover from these situations.

`try` wraps the risky code, `except` catches a specific exception type and defines a fallback, `else` runs only when nothing went wrong, and `finally` runs unconditionally — the natural place to release a resource or close a connection. `raise` deliberately throws an exception, including custom ones. Catching a bare `except:` with no type attached is worth avoiding, since it silently swallows everything, including bugs unrelated to the situation being handled.

```
try:
    val = int(data)
except ValueError:
    val = 0
else:
    print("conversion succeeded")
finally:
    print("done, no matter what")
```

## 6. NumPy

---

A plain Python list stores each element as a separate object scattered in memory, with its own type overhead. NumPy's `ndarray` instead stores a fixed-type block of numbers contiguously and pushes the actual looping into compiled C code, so operations run as a single vectorized pass rather than an interpreted loop. This is why NumPy underlies pandas, scikit-learn, and most of the numerical Python ecosystem.

Broadcasting lets NumPy apply an operation across arrays of different shapes without an explicit loop — adding a number to every element, for example, without ever writing `for x in array`.

```
import numpy as np
marks = np.array([70, 80, 90])
mean_val = np.mean(marks) # 80.0
scaled = marks * 1.1      # broadcasting, no loop needed
```

## 7. Pandas

---

Pandas builds on NumPy to offer two structures suited to real data: a Series (one labeled column) and a DataFrame (a table made of Series sharing an index). Day-to-day work — filtering rows, grouping and aggregating, merging datasets on a key, cleaning missing values — comes down to a small recurring set of operations: boolean masking, `.groupby()`, `.merge()`, and `.fillna()/dropna()`.

`.loc[]` and `.iloc[]` are the two ways to slice a DataFrame, and confusing them is a common bug source: `.loc` selects by label (the index or column name), while `.iloc` selects by integer position, the same way list indexing works.

```
import pandas as pd
df = pd.read_csv("emp.csv")
high_pay = df[df["salary"] > 50000]
by_dept = df.groupby("department")["salary"].mean()
```

## 8. Selenium Automation

---

Selenium drives an actual browser through the WebDriver API — clicking, typing, scrolling — making it the right tool for pages whose content is rendered by JavaScript after the initial load, where a plain HTTP request would see nothing useful.

Timing is the recurring challenge. An implicit wait polls for up to N seconds whenever an element can't be found, applied globally. An explicit wait is more precise, waiting for one specific condition (an element becoming clickable, or text appearing) before continuing — usually the more reliable choice on pages with asynchronous rendering. Where a site exposes an official REST API for the needed data, that should be used instead of scraping the rendered page — it is faster, less brittle to layout changes, and avoids terms-of-service risk.

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

element = WebDriverWait(driver, 10).until(
    EC.element_to_be_clickable((By.ID, "submit-btn"))
)
```

## 9. Regular Expressions

---

A regular expression describes a pattern of text rather than a literal string, and Python's `re` module turns that pattern into a search, match, or substitution. The core building blocks are `\d` for a digit, `\w` for a word character, `+` for one or more, `*` for zero or more, and parentheses to capture a group separately from the overall match.

Regex is genuinely good at validating formats — emails, phone numbers, IDs — and pulling structured fragments from unstructured text. It is a poor tool for anything with real nested structure, such as HTML

or JSON; a dedicated parser handles nesting correctly, while a regex tuned to "mostly work" on HTML tends to break as soon as the markup grows slightly more complex than the sample it was tested against.

```
import re
email_pattern = r"[\w.-]+@[ \w.-]+\.\w+"
match = re.search(email_pattern, text)
if match:
    print(match.group())
```

## 10. Multithreading, Multiprocessing, and Concurrency vs Parallelism

---

Concurrency and parallelism are often used interchangeably but describe different things. Concurrency means handling several tasks over the same period, switching between them — one cook alternating between four dishes. Parallelism means multiple tasks genuinely running at the same instant — four cooks, each on one dish. A single-core CPU can be concurrent but never truly parallel; a multi-core machine can be both.

Multithreading runs several threads inside one process, sharing memory, which suits I/O-bound work such as waiting on a network call or disk read, where the CPU is mostly idle anyway. In CPython, the Global Interpreter Lock (GIL) prevents two threads from executing Python bytecode at the same instant, which is why threading gives little benefit for CPU-bound work like heavy computation.

Multiprocessing sidesteps the GIL by spawning separate processes, each with its own memory and interpreter, allowing genuine parallel execution across CPU cores. The trade-off is heavier overhead, since processes don't share memory automatically and passing data between them costs more than between threads.

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

# I/O-bound: many small network calls waiting on a response
with ThreadPoolExecutor(max_workers=8) as pool:
    results = list(pool.map(fetch_url, urls))

# CPU-bound: heavy computation that needs real parallel cores
with ProcessPoolExecutor() as pool:
    results = list(pool.map(crunch_numbers, chunks))
```

## 11. Software Development Life Cycle (SDLC)

---

SDLC is the structured sequence software moves through from idea to retirement: requirements gathering, design, coding, testing, deployment, and maintenance. It doesn't prescribe a specific process — Waterfall versus Agile is a separate decision — it is simply the map of phases that any process still has to pass through.

Maintenance is often underestimated, yet most of a system's lifetime cost comes from the years of bug fixes, security patches, and small additions after deployment, not from writing the first version. Skipping

proper design or testing early doesn't remove that cost — it just moves it later and makes it more expensive.

## 12. Agile Philosophy

---

Agile is a mindset before it is a set of ceremonies. Its core bet is that requirements will change once people see working software, so it is better to deliver something small and real early, gather feedback, and adjust, rather than design a complete system upfront that might already be wrong by the time it ships.

In practice this means short delivery cycles, close collaboration with whoever the software is for, and treating a changing requirement as expected rather than as a planning failure. Scrum, Kanban, and similar frameworks are specific implementations of this philosophy, not the philosophy itself.

## 13. Scrum Framework

---

Scrum is one concrete way to run Agile. Work is broken into a backlog, and a fixed-length Sprint (commonly two to four weeks) pulls a chunk of that backlog to build. Three roles keep it running: the Product Owner decides what gets built and in what order, the Scrum Master keeps the process unblocked, and the Developers do the building.

The ceremonies solve specific coordination problems rather than filling a calendar: the Daily Standup surfaces blockers early, the Sprint Review shows real working software to stakeholders, and the Retrospective is where the team reflects on how it worked together — often the first ceremony to be skipped, and the one that costs the most over time.

## 14. Git Version Control

---

Git tracks changes as a graph of commits rather than a single linear history, which is what makes branching cheap — a feature branch can be started off main, worked on in isolation, and merged back once ready, without risking the stable history in the meantime.

A Pull Request is the review checkpoint between finishing a branch and merging it into the main codebase — where a teammate reads the diff, runs CI checks, and either approves it or requests changes. Skipping that step is how small bugs quietly become production incidents.

```
git checkout -b feature-login
git add .
git commit -m "Add login form validation"
git push origin feature-login
# open a Pull Request from feature-login into main
```

## 15. Documentation Standards

---

A README is the first thing anyone — including a future version of the original author — reads before running a project. At minimum it should explain what the project does, how to install it, how to configure it, and how to use it, including setup steps, environment variables, and, for anything exposing an API, the interface itself.

Good documentation is written for a reader who knows nothing yet, not for the author who already understands the system — which is exactly why it tends to go stale unless updating it is treated as part of finishing a feature rather than an afterthought.

## 16. Risk Management

---

Risk management in software is structured pessimism done early enough to be useful. Every meaningful risk is scored on two axes — likelihood and impact — and the highest-scoring risks get a mitigation plan before they become real incidents.

A risk that is high-impact but low-probability, such as a data-center outage, still needs a plan, usually a cheap standing one like automated backups. A risk that is high-probability but low-impact might simply be accepted and monitored, since the cost of prevention could outweigh the cost of the problem itself.

## 17. Coding Standards

---

Coding standards are the agreed rules a team follows for naming, file structure, and error handling so that code written by several people still reads as if it came from one codebase. The value isn't in any single rule being objectively correct — it's in everyone following the same one, which is what makes code review fast and onboarding easier.

In a multi-contributor project, this matters more than it does solo: without an agreed standard, every pull request turns into a debate about style instead of substance.

## 18. PEP 8

---

PEP 8 is Python's official style guide, and it is specific rather than vague: four spaces per indentation level, a line length capped around 79–88 characters depending on the tool, `snake_case` for functions and variables, `PascalCase` for classes, and `UPPER_CASE` for constants.

Following it isn't about aesthetics — it lets any Python developer read unfamiliar code at normal speed instead of pausing to decode someone's personal convention. Tools like Black and Ruff exist largely to enforce PEP 8 automatically, removing the need to argue about it in review.

## 19. Docstrings vs Comments

---

A comment is for the person reading the source code right now, explaining a specific line or decision inline. A docstring is different: a triple-quoted string placed right after a module, class, or function definition, documenting the public interface — what the thing does, what it expects, and what it returns.

The practical difference is that a docstring is inspectable at runtime — `help(my_function)` reads it directly, and documentation generators like Sphinx can build entire reference sites from docstrings alone. A comment, by contrast, is invisible to anything except someone reading the raw source file.

```
def calc_net(gross, tax=0.2):
    """Return take-home pay after applying a flat tax rate.

    gross -- pre-tax amount
    tax -- decimal tax rate, default 0.2
    """
    return gross * (1 - tax) # explicit, not compound assignment
```

## 20. Logging and Auditing

---

`print()` is fine for a quick script but doesn't scale — it can't be turned off in production, carries no severity level, and writes nothing durable once the terminal closes. Python's logging module solves all three: messages carry a severity (DEBUG, INFO, WARNING, ERROR, CRITICAL), and output can be filtered, formatted, and routed to a file or monitoring service without touching the calling code.

One rule is non-negotiable: never log passwords, API keys, tokens, or other sensitive values. Log files often end up with looser access control than the database itself, so a secret that leaks into a log becomes effectively public inside the organization.

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("App booted")
logging.warning("Retry limit approaching for job %s", job_id)
logging.error("Payment failed", exc_info=True)
```

## 21. Writing Efficient Code

---

Efficient code usually comes down to picking the right data structure before reaching for clever tricks. Checking whether a value exists in a Python list takes time proportional to the list's size, since each element has to be checked in turn; the same check against a set or dict key is close to constant time regardless of size, because both are backed by a hash table.

The habit worth building early is to profile before optimizing. It's tempting to guess which part of a program is slow and rewrite it, but the real bottleneck is often somewhere unexpected — a database query run in a loop, not the surrounding Python logic. Tools like cProfile show where time is actually going before effort is spent optimizing the wrong thing.

```
# O(n) lookup – checks every element in the worst case
if user_id in user_list: ...

# O(1) average lookup – hash table under the hood
if user_id in user_id_set: ...
```

## 22. Architecture Principles — DRY, KISS, YAGNI, SOLID

---

DRY (Don't Repeat Yourself) holds that the same piece of logic shouldn't exist in two places, because the moment it needs to change, every copy has to be remembered and updated, and eventually one gets missed.

KISS (Keep It Simple) and YAGNI (You Aren't Gonna Need It) pull in the same direction: build what is actually required now, not the flexible, general-purpose version that might be needed someday.

Speculative abstraction adds real complexity today in exchange for a benefit that, most of the time, never arrives.

SOLID is a set of five design guidelines for object-oriented code: Single Responsibility (a class should have one reason to change), Open/Closed (open for extension, closed for modifying working code), Liskov Substitution (a subclass should work anywhere its parent is expected), Interface Segregation (don't force a class to implement methods it doesn't need), and Dependency Inversion (depend on abstractions rather than concrete implementations). Together they push toward code that can be tested and changed one piece at a time.

### Interview / Practice Angle

- *Point to a place in a project where DRY was violated — what would refactoring it actually look like?*
- *Give an example of a YAGNI violation — a feature or abstraction built "just in case" that never got used.*

## 23. Unit Testing

---

A unit test checks one small piece of behaviour in isolation — usually a single function — asserting that a given input produces the expected output, independent of databases, networks, or other parts of the system. If a test depends on a live database being in a particular state, it stops being a fast, reliable unit test and becomes closer to an integration test.

A common pattern worth internalizing is Arrange-Act-Assert: set up the inputs (Arrange), call the function under test (Act), then check the result (Assert). Tests typically run automatically in a CI pipeline on every push, so a broken change is caught within minutes rather than surfacing in production days or weeks later.

```
def add(a, b):
    return a + b
```

```
def test_add():  
    # Arrange  
    a, b = 2, 3  
    # Act  
    result = add(a, b)  
    # Assert  
    assert result == 5
```

## 24. Ruff Linter

---

Ruff is a static analyzer written in Rust, built to be dramatically faster than the older Python-based tools it has largely replaced, such as Flake8 and isort. It doesn't run the code — it reads it and flags problems: unused imports, dead code, style violations, and likely bugs, all before anything executes.

Running `ruff check --fix .` both reports issues and auto-fixes the ones safe to fix mechanically, which is why it has become a standard pre-commit or CI step, catching an entire class of small mistakes before a human reviewer even opens the pull request.

```
ruff check --fix .
```

## 25. Black Formatter

---

Black is deliberately opinionated, with almost no configuration options — that is the entire point. Instead of a team debating tabs versus spaces or trailing-comma placement, Black reformats code the same way every time, for every contributor, and the debate simply stops being available to have.

Running it is a single command, and most teams wire it into a pre-commit hook so code is auto-formatted before it is even committed. That means pull requests show the actual logic change in the diff, not a wall of whitespace noise from someone's editor reformatting the whole file.

```
black .
```